

# Esame giugno

## Gestore affidabilità → Ripresa a caldo

### ESERCIZI

#### d) (4) Gestore dell'affidabilità

Si supponga che si verifichi un guasto di sistema con perdita di contenuto della memoria centrale. Al momento del guasto il contenuto del file di LOG è il seguente:

B(T1), B(T2), U(T1,O1,B1,A1), I(T2,O2,A2), B(T3), B(T4), U(T4,O2,B3,A3), U(T3,O1,B4,A4), A(T1), CK(T2,T3,T4), C(T4), B(T5), U(T3,O3,B5,A5), U(T5,O4,B6,A6), D(T2,O5,B7), C(T2), A(T5), I(T3,O6,A8) guasto

Illustrare cosa accade al riavvio del sistema nell'esecuzione della procedura di ripresa a caldo.

#### 1 Risalire fino all'ultimo checkpoint

$$\text{Undo} = \{T_2, T_3, T_4\} \quad \text{Redo} = \emptyset$$

#### 2 In avanti dal checkpoint fino al guasto

$$\text{Undo} = \{\cancel{T_2}, \cancel{T_3}, \cancel{T_4}, T_5\} \quad \text{Redo} = \{T_4, T_2\}$$

$$\text{Undo} = \{T_3, T_5\} \quad \text{Redo} = \{T_4, T_2\}$$

#### 3 All'indietro per disfare le transazioni in undo

$$\text{Undo} \quad I(T_3, O_6, A_8) \Rightarrow \text{Delete}(O_6)$$

$$U(T_5, O_4, B_6, A_6) \Rightarrow O_4 := B_6$$

$$U(T_3, O_3, B_5, A_5) \Rightarrow O_3 := B_5$$

$$U(T_3, O_1, B_4, A_4) \Rightarrow O_1 := B_4$$

#### 4 In avanti per rifare le transazioni in redo

$$\text{Redo} \quad I(T_2, O_2, A_2) \Rightarrow O_2 := A_2$$

$$U(T_4, O_2, B_3, A_3) \Rightarrow O_2 := A_3$$

$$D(T_2, O_5, B_7) \Rightarrow \text{Delete}(O_5)$$

#### e) Esecuzione concorrente

Dato il seguente schedule S:

S: r2(y), w3(z), r2(x), w1(z), w2(y), w3(t), r3(y), w3(x), r1(x), r1(y), w0(x), w1(t)

- (2) Indicare se S è view-serializzabile (VSR) oppure no (giustificare la risposta)
- (2) Indicare se S è conflict-serializzabile (CSR) oppure no (giustificare la risposta)
- (2) Indicare se S è 2PL (vale a dire, rispetta la regola di serializzabilità del locking a due fasi)

- View-serializzabilità: Bisogna calcolare l'insieme Legge\_da e l'insieme Scritture\_finali:

$$\text{Legge\_da}(S) = \{(r_3(y), w_2(y)), (r_1(x), w_3(x)), (r_1(y), w_2(y))\}$$

$$\text{Scritture\_finali}(S) = \{w_1(t), w_0(x), w_2(y), w_1(z)\}$$

Controlliamo se questo schedule è view-serializzabile, cioè se esiste uno schedule seriale con insiemi legge\_da e scritture\_finali equivalenti a quelli di S

$$\text{Legge\_da}(S) = \{(r_3(y), w_2(y)), (r_1(x), w_3(x)), (r_1(y), w_2(y))\}$$

$$\text{Scritture\_Finali}(S) = \{w_1(t), w_0(x), w_2(y), w_1(z)\}$$

$t_2 < t_3$        $t_3 < t_1$        $t_2 < t_1$   
 $t_3 < t_1$        $t_3 < t_0$        $t_3 < t_1$

Si possono inoltre guardare le letture che non compaiono nelle scritture\_finali. Si nota che la scrittura  $r_1(x)$  non legge da  $w_0(x)$ , quindi  $t_1 < t_0$  perchè altrimenti  $(r_1(x), w_0(x))$  apparterebbe a legge\_da e non ci serve.

$t_2 < t_0$  perchè altrimenti  $(r_2(x), w_0(x))$  apparterebbe all'insieme legge\_da e non serve.

$t_2 < t_3$  perchè altrimenti  $(r_2(x), w_3(x))$  apparterebbe all'insieme legge\_da e non serve

Notiamo che:  $t_2 < t_3 < t_1 < t_0$

$$S_R = t_2 \ t_3 \ t_1 \ t_0$$

$$= r_2(y), r_2(x), w_2(y), w_3(z), w_3(t), r_3(y), w_3(x),$$

$$w_1(z), r_1(x), r_1(y), w_1(t), w_0(x)$$

$$\text{Legge\_da}(S_R) = \{(r_3(y), w_2(y)), (r_1(x), w_3(x)), (r_1(y), w_2(y))\}$$

$$\text{Scritture\_Finali}(S_R) = \{w_0(x), w_2(y), w_1(z), w_1(t)\}$$

Abbiamo determinato che S è view-equivalente a  $S_R$ , quindi è view-serializzabile

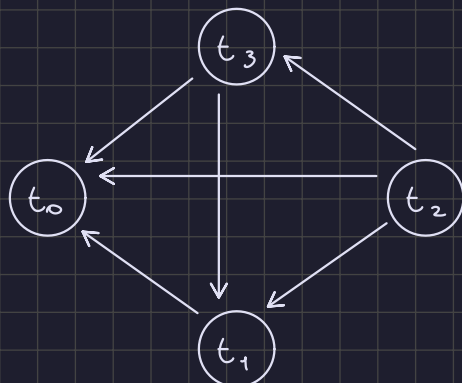
- Conflict-serializzabilità: si controlla se il grafo dei conflitti è aciclico. Un conflitto è una coppia di operazioni fatte sulla stessa risorsa da transazioni diverse e una delle due deve essere di scrittura.

S:  $r_2(y), w_3(z), r_2(x), w_1(z), w_2(y), w_3(t), r_3(y), w_3(x), r_1(x), r_1(y), w_0(x), w_1(t)$

$$\text{Conflitti} = \{(w_3(z), w_1(z)), (r_2(x), w_3(x)), (r_2(x), w_0(x)),$$

$$(w_2(y), r_3(y)), (w_2(y), r_1(y)), (w_3(t), w_1(t)),$$

$$(w_3(x), r_1(x)), (w_3(x), w_0(x)), (r_1(x), w_0(x))\}$$



Siccome il grafo è aciclico con il seguente ordinamento topologico:

$t_2 \ t_3 \ t_1 \ t_0$

lo schedule S è conflict-serializzabile, e quindi è anche view-serializzabile perchè  $CSR \subset VSR$

- S è 2PL

S: r2(y), w3(z), r2(x), w1(z), w2(y), w3(t), r3(y), w3(x), r1(x), r1(y), w0(x), w1(t)

$r_2(y)$  t2 fa r\_lock(y)  
 $w_3(z)$  t3 fa w\_lock(z)  
 $r_2(x)$  t2 fa r\_lock(x)  
 $w_1(z)$  t1 aspetta che t3 rilasci il w\_lock(z) e quando lo fa t1 esegue w\_lock(z). t3 esegue un unlock e inizia la fase discendente, ma siccome t3 esegue un w\_lock(x) successivamente, non può acquisire il lock e di conseguenza S non è 2PL

Lo schedule S non rispetta i vincoli per il 2PL

#### f) Ottimizzazione

Si consideri il seguente schema relazionale contenente i risultati di elezioni :

**STUDENTE**(Matricola, Nome, Cognome, Indirizzo, Comune, Provincia, Regione, Email);

**ESAME**(Studente, Docente, Data, Oralnizio, Durata, Voto)

**DOCENTE**(CodiceFiscale, Cognome, Nome, Qualifica)

Vincoli d'integrità referenziale: ESAME.Studente -> STUDENTE, ESAME.Docente -> DOCENTE

Data la seguente interrogazione

```
SELECT S.Matricola, S.Cognome, S.Nome, E.Docente, E.Data, E.Oralnizio, E.Voto
FROM STUDENTE S JOIN ESAME E ON (S.Matricola = E.Studente)
WHERE S.Provincia = 'Verona' AND E.Data > '01/01/2023'
```

(3) Calcolare il costo dell'interrogazione in termini di numero di accessi a memoria secondaria sotto le seguenti ipotesi:

- la selezione degli studenti viene eseguita attraverso una scansione della tabella STUDENTE, il risultato della selezione viene mantenuto nel buffer
- la selezione degli esami viene eseguita con una scansione sequenziale della tabella ESAME, il risultato viene salvato in 1750 pagine della memoria secondaria (201500 righe).
- l'ordine di esecuzione del join è STUDENTE  $\bowtie$  ESAME e le operazioni di join vengono eseguite con la tecnica "Nested Loop Join" con una pagina di buffer disponibile per ogni tabella
- NP(DOCENTE) = 14, NP(ESAME) = 2500, NP(STUDENTE) = 250  
NR(DOCENTE) = 1700, NR(ESAME) = 294500, NR(STUDENTE) = 15500
- VAL(Provincia, STUDENTE) = 10, VAL(Studente, ESAME) = 15500

Le operazioni sono nell'ordine:

1. Selezione su provincia
2. Selezione su data
3. Join

1. Selezione su provincia = lettura + scrittura

perchè mantenuto nel buffer

$$NP(\text{studente}) + 0$$

2. Selezione su data = lettura + scrittura

$$NP(\text{Esame}) = 2500 + 1750 = 4250$$

perchè il risultato viene salvato in 1750 pagine di memoria secondaria (201500 righe)

3. Join R  $\bowtie$  S

$$NP(R) + NR(R) \cdot NP(S) =$$

Studenti:  
con selezione  
su provincia

Esami  
con selezione  
sulla data

$$= 0 + \frac{NR(\text{Studente})}{Val(\text{Provincia}, \text{Studente})} \cdot 1750$$

Già nel buffer

$$= 0 + \frac{15500}{10} \cdot 1750$$

$$= 0 + 1550 \cdot 1750 = 2625000$$

Il costo totale della query è la somma delle singole operazioni:

$$250 + 4250 + 2625000 = 2629500$$

(2) Come cambia il costo della query considerando la presenza di un indice B+-tree di profondità 3 sull'attributo Studente di ESAME.

L'aggiunta dell'indice cambia il costo soltanto della join, quindi il costo delle altre operazioni rimane invariato.

3. Studente join Esame, bisogna caricare la selezione su studente, che costa 0 perchè è già nel buffer, e bisogna anche caricare la selezione su Esame, quindi il costo è:

PI=3

$$NP(\text{Sel. Verona}(\text{Studente})) + NR(\text{Sel. Verona}(\text{Studente})) \cdot \left( PI + \frac{NR(\text{Sel. Data}(\text{Esame}))}{Val(\text{Studente}, \text{Esame})} \right)$$

Bisogna calcolare la selettività di esame sull'indice:

$$\frac{NR(\text{Sel. Data}(\text{Esame}))}{Val(\text{Studente}, \text{Esame})} = \frac{201500}{15500} = 13$$

Quindi il costo finale della join è:

$$0 + 1550 \cdot (3 + 13) = 1550 \cdot 16 = 24800$$

Il costo totale della query con l'indice è:

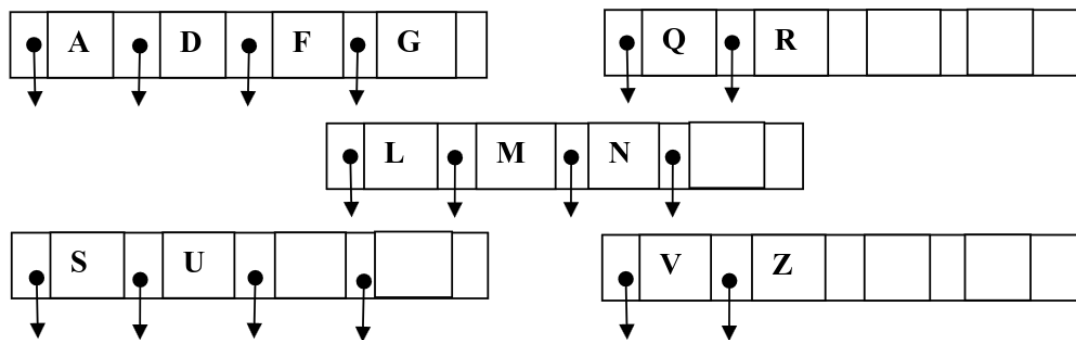
$$250 + 4250 + 24800 = 29300$$

Quindi l'indice migliora significativamente il costo

g) B+-tree

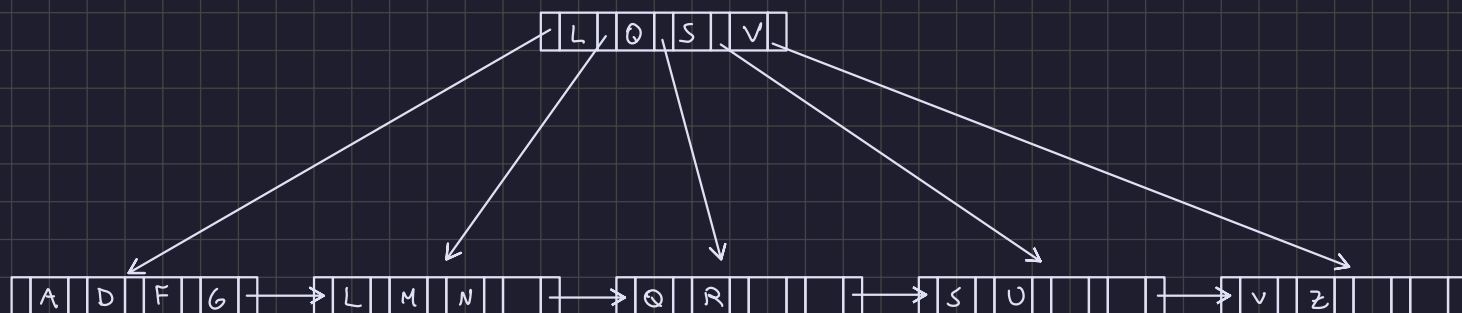
Data la seguente lista di possibili valori chiave  $L = (A, B, C, D, E, F, G, H, I, L, M, N, O, P, Q, R, S, T, U, V, Z)$ :

a) (1) costruire un B+-tree (fan-out=5) che contenga i seguenti nodi foglia:

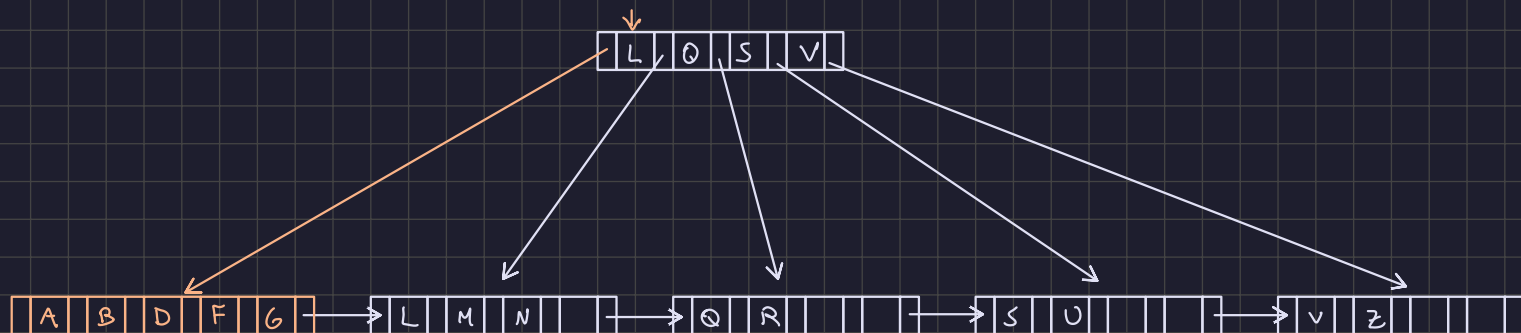


Vincoli di chiave:  $2 \leq \# \text{chiavi} \leq 4$

Vincoli di puntatori:  $3 \leq \# \text{puntatori} \leq 5$

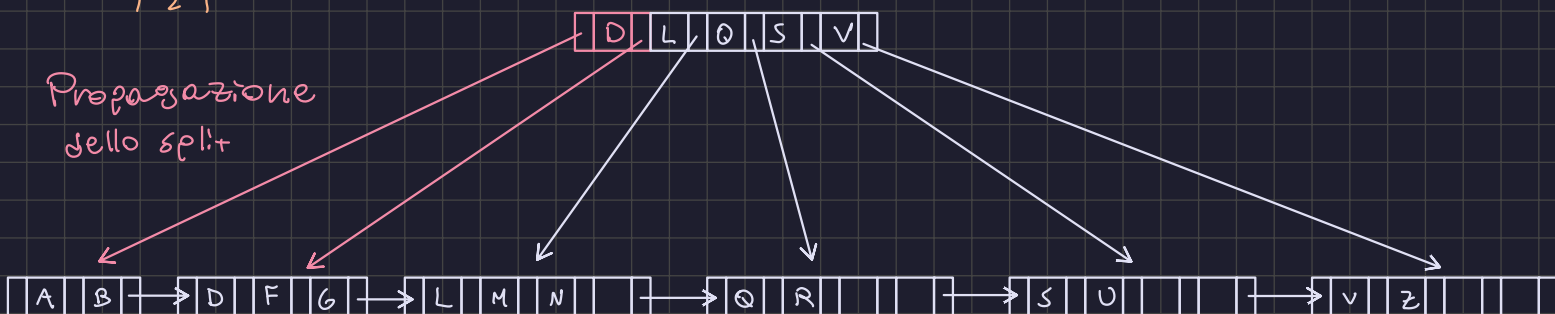


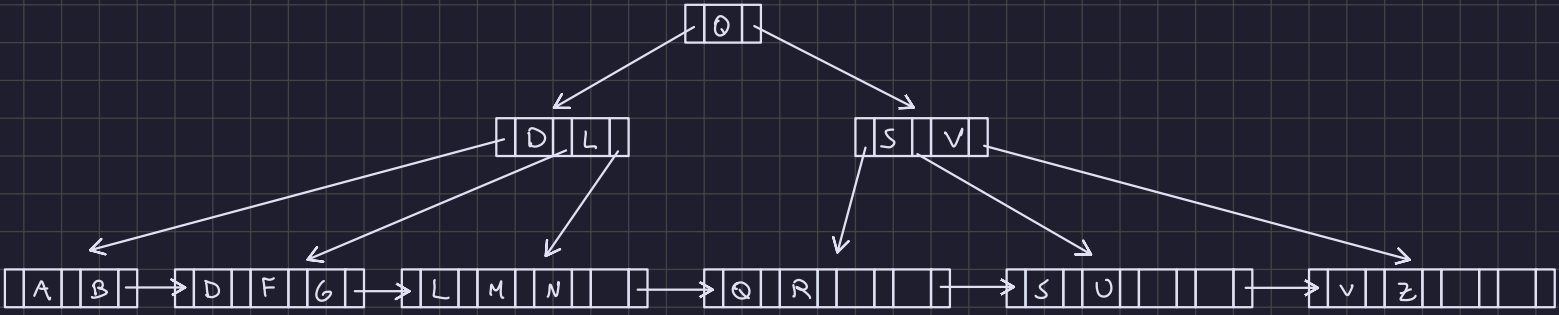
b) (2) mostrare l'albero dopo l'inserimento del valore chiave B



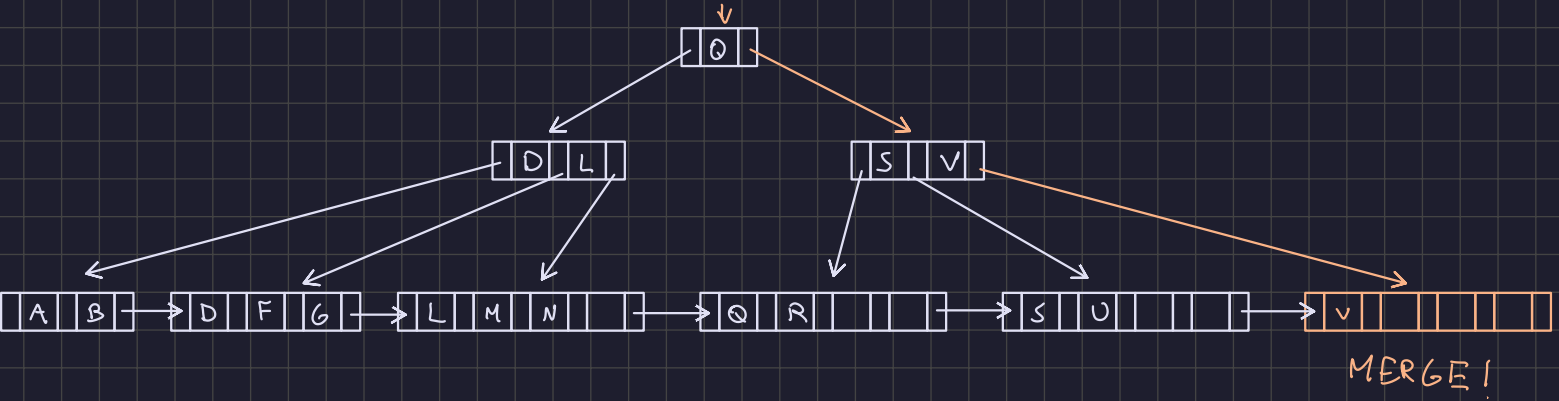
SPLIT!  
 $\lfloor \frac{n-1}{2} \rfloor$

Propagazione  
dello split





c) (2) partendo dal risultato del punto precedente mostrare l'albero dopo la rimozione del valore chiave Z.



Propagazione del  
merge  
[Q, S]

